
Returning to Development States: When Full Sandbox Snapshots Help Coding Agents

Daniel Thi Graviet¹ Tanmay Shah²

Daytona, San Francisco, California, USA

Software Engineering, University of Waterloo, Waterloo, Ontario, Canada
dgraviet@daytona.io

Abstract

Coding agents often make useful progress before they fail, but retry-from-scratch discards the machine state created along the way. This makes failure recovery an interesting search problem: if intermediate environment states preserve meaningful progress, restoring and branching from them could outperform repeatedly restarting from the initial state. We implement Go-Explore-style search over Daytona snapshots, finding that benefits depend critically on task complexity and the reconstructibility of progress. Our results suggest that environment-level search is most valuable when useful progress lives outside easily reconstructed source state, motivating snapshot-aware agents that preserve and revisit high-value execution states during long-horizon coding tasks.

1 Introduction

A coding-agent failure is rarely a clean reset point. Consider an agent debugging a repository. It installs missing dependencies, reproduces the failing test, modifies several files, and creates a local database needed by the application. It then makes one bad edit and fails the task. A standard retry starts again from the original repository, discarding not only the bad edit but also every useful change to the machine that came before it.

Most test-time scaling methods operate above this machine state. Independent retries sample new trajectories from the initial environment. Memory systems preserve text. Search methods preserve source-level state such as commits and file context. None of these necessarily preserve the complete execution environment, including installed packages, running services, generated artifacts, databases, caches, and other state accumulated during interaction.

Go-Explore (Ecoffet et al., 2021) offers a natural alternative. It remembers promising states, returns to them, and explores again. We apply this idea to cod-

ing agents by treating a full Daytona sandbox snapshot as the searchable state. A snapshot freezes the live machine so that a new agent can resume from an intermediate point rather than reconstructing it from the repository or a textual summary.

We initially hypothesized that branching from promising snapshots would solve more tasks than independent retry under a fixed token budget. Our experiments refute that broad hypothesis on the Terminal-Bench (The Terminal-Bench Team, 2025) tasks we tested, but they also reveal when environment-level state matters and what prevents snapshot search from being effective today.

This paper makes four contributions:

- We show that full sandbox snapshots are a reliable restore primitive for coding-agent search. Across four Terminal-Bench anchor tasks, 24/24 restored continuations reproduced a valid live state, enabling agents to resume computation from intermediate trajectories (Section 5).
- We provide a matched-budget comparison between snapshot branching and independent retry. On our pre-registered harder Terminal-Bench subset, retry solves 17/25 tasks while snapshot-based search solves 6/25, establishing that reliable restoration alone is not sufficient to improve test-time scaling (Section 6).
- We identify concrete failure modes of

¹Daytona, San Francisco, California, USA ²Software Engineering, University of Waterloo, Waterloo, Ontario, Canada. Correspondence to: Daniel Thi Graviet <dgraviet@daytona.io>.

environment-level search. Snapshot branching loses through budget fragmentation, selection of misleading intermediate states, agent handoff cost, and weak signals for deciding which states deserve further exploration (Section 8).

- We show that the value of snapshots depends on what information the environment contains. On extract-elf, compressed source-level state can outperform a full snapshot, while on a custom task whose progress lives in a SQLite database outside git, full snapshots solve 4/5 attempts and source-only restoration solves 0/5 (Section 7). These results suggest that environment-level search is most useful when progress cannot be reconstructed from repository state alone.

2 The Problem

A coding-agent trajectory changes more than source files. During execution, the agent may install packages, generate artifacts, populate databases, modify configuration, start services, or otherwise alter the machine in ways that affect future actions. A retry from the original repository loses all of this state, while a source-level checkpoint such as a git diff preserves only the subset represented in files under version control. This creates a simple question. When an agent makes useful progress before failing, what state must we preserve to make that progress reusable? If source state is sufficient, simpler checkpoints suffice. But if critical progress lives in the execution environment itself, such as installed packages, service states, and runtime artifacts, full-machine snapshots become necessary. They uniquely preserve this state without the cost of re-execution, making them valuable for long-horizon tasks where such progress accumulates.

3 The Idea

The core idea is simple. Instead of treating a failed coding-agent run as one indivisible attempt, treat it as a path through a sequence of machine states. As the agent works, some states are more promising than others. Perhaps the bug has been reproduced, the right dependency is installed, or most of the fix is already in place. We periodically snapshot those states and store them in an archive. When a trajectory fails, we do not have to restart from the beginning. We can restore one of the archived states, place a new agent there, and continue exploring from that point. In this view, a sandbox snapshot plays the same role that a saved game does in search. It lets us return to a useful intermediate position and try a different continuation without replaying everything that came before. The

important distinction is that the saved state is the entire execution environment, not just the agent’s text history or a patch. This gives the search procedure access to any progress encoded in the live machine.

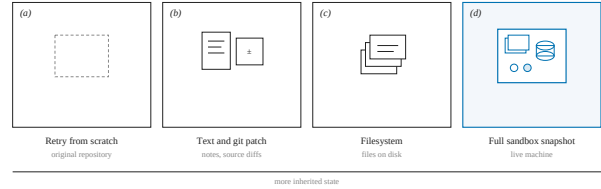


Figure 1. What the next attempt starts from. (a) Retry from scratch uses the original repository. (b) Text and git patches keep notes and source diffs. (c) A filesystem checkpoint keeps files on disk. (d) A full sandbox snapshot keeps the live machine. We study (d) as a restore primitive.

4 System and Evaluation

We evaluate two questions. First, can a coding agent reliably resume from an intermediate machine state? Second, does returning to such states improve task success compared with spending the same inference budget on independent retries?

4.1 Snapshot-based Search

Our system periodically saves promising points during a coding-agent run. For example, it may save a snapshot after the agent edits files, runs tests, or makes other useful progress.

Each snapshot captures the full Daytona sandbox at that moment. We store these snapshots in an archive and rank them based on simple signals such as file edits and test activity. This simple ranking provides a foundation that future work could extend with more sophisticated signals, learned models, or domain-specific heuristics.

In practice, when an agent fails on a hard task, restarting from the beginning loses all intermediate progress. Instead, our system identifies a promising saved state from the failed run, restores the sandbox to that point, and launches a new child trajectory. The child inherits the restored machine state but receives a fresh agent context, allowing a different approach from a promising midpoint.

This is the core Go-Explore idea. Instead of always restarting from the beginning, we return to a promising state and explore a different continuation from there.

We compare this against independent retry, where the

same total token budget is spent on fresh attempts that all start from the original task state. Both methods use the same model and the same total inference budget. The main difference is whether later attempts start over or continue from saved progress.

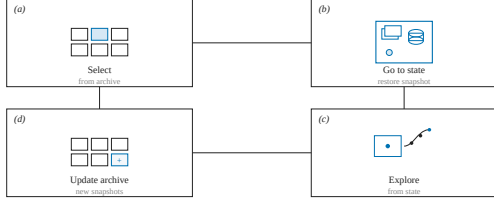


Figure 2. Search over Daytona sandboxes. (a) Select a snapshot from the archive. (b) Restore that sandbox (Go-Explore’s “go to state”). (c) A new agent explores from the restored machine. (d) New snapshots update the archive. Independent retry is the matched-budget baseline and is not shown.

4.2 Experimental Setup

Unless otherwise noted, all experiments use Claude Haiku 4.5 on Terminal-Bench 2.0 (The Terminal-Bench Team, 2025) through Harbor (Harbor Framework Team, 2026) and Daytona.¹ We intentionally use a lower-cost model because this work focuses on token-efficient agent search under fixed budgets. Models in this cost range are also practical for large numbers of rollouts in agent harnesses, where repeated attempts can quickly become expensive.

Our headline comparison uses five tasks selected for intermediate single-attempt difficulty. For each task, we run five seeds with a total budget of one million tokens per seed. Independent retry spends that budget across three fresh attempts, while snapshot search uses one root trajectory followed by up to two restored children. The task set and protocol were fixed before examining the final comparison.

5 Restore Reliability

Before evaluating search, we first test whether restoration itself works.

We select four Terminal-Bench tasks with high baseline solve rates and run three root trajectories per task. From these roots we launch 24 restored continuations.

All 24/24 continuations successfully restored their intended machine state. The fidelity extends beyond ordinary source files. In one fix-git example, restoring

the sandbox reproduced an unresolved merge together with the corresponding Git index state.

Restored continuations therefore remain capable of solving the task after resuming from an intermediate environment. The experiment establishes restoration as a usable primitive but it does not establish an advantage over retry.

6 Snapshot Search vs Retry

We next test whether branching from promising intermediate states can outperform repeated fresh attempts under the same total token budget.

On this Terminal-Bench subset, independent retry performs better overall than snapshot branching. Retry wins on every task and solves nearly three times as many task-seed pairs overall.

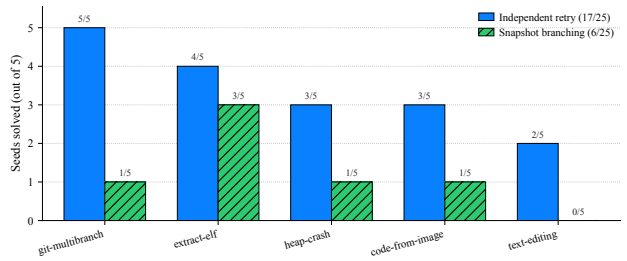


Figure 3. Pre-registered headline comparison. Five file-centric Terminal-Bench tasks $\times$ five seeds, 1,000,000 tokens. Independent retry from scratch outperforms snapshot branching (17/25 versus 6/25). When useful progress is mostly in the filesystem, starting over is often enough. This total is frozen; later warehouse rows are not pooled into it.

Table 1. Headline solve rates on the pre-registered task set (five seeds per task, 1,000,000 tokens per seed).

Task	Retry	Branch
git-multibranch	5/5	1/5
extract-elf	4/5	3/5
custom-memory-heap-crash	3/5	1/5
code-from-image	3/5	1/5
large-scale-text-editing	2/5	0/5
Total	17/25 (68%)	6/25 (24%)

However, snapshot search is not simply failing to restore useful computation. We observe four cases where the root trajectory fails but a child restored from that root later succeeds. Three occur on extract-elf and one on custom-memory-heap-crash. In a separate follow-up experiment, we specifically tested whether snapshot branching could recover from failed roots. On

¹<https://www.daytona.io/>

extract-elf, snapshot branching successfully recovered 67% of failed roots, substantially higher than independent retry’s overall success rate of 38% on that task. On regex-log, the recovery rate was more modest at 13%. This shows that snapshot branching is effective at salvaging failed trajectories when given the opportunity, but in the main experiment, such recoveries were too infrequent to offset the total cost of searching, restoring, and continuing from snapshots.

The mechanism therefore works in the narrow sense we intended. A failed trajectory can contain a state from which another agent succeeds. The problem is that, on the Terminal-Bench tasks we tested, these recoveries are not frequent enough to offset the cost of finding and exploiting those states.

7 What State Is Worth Preserving?

The headline experiment asks whether full snapshots improve search. A separate question is whether the full machine state itself contains information worth preserving.

We test this by comparing different ways of saving the same intermediate progress.

On extract-elf, saving the changed files is often enough. From the same checkpoint and with the same remaining token budget, a child started from a Git diff or from replayed commands can succeed even when the child started from the full snapshot runs out of budget. For this kind of task, most useful progress is already captured in the repository, so restoring the entire machine adds little.

We then construct staged-service-repair, where the agent’s progress depends on a SQLite database at /var/lib/inventory/store.db that must exist for the verifier to pass. This reflects a common production scenario: agents modify system state such as databases, caches, and services that version control systems never track. A Git diff captures only code changes, not this underlying execution state, making full snapshots essential.

We freeze a Daytona snapshot partway through the task and compare three ways of starting the next agent, all with the same remaining token budget of 200,000:

- Clean restart: Start from the original task state with no prior progress.
- Git diff applied: Restart with the saved Git diff restored, but without the database state.
- Full snapshot restore: Restore the complete Daytona snapshot, including both code changes and database state.

At this checkpoint, the Git diff is empty because the code change has already been committed, while the state required by the verifier exists only in the database.

Starting from that same point, we compare the following conditions.

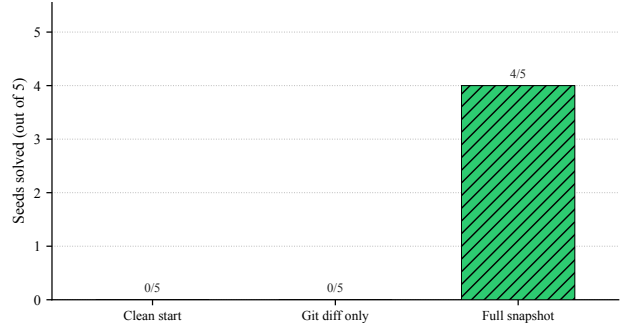
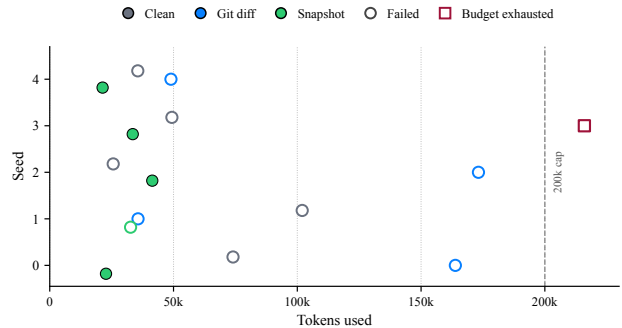


Figure 4. staged-service-repair, same planted checkpoint, 200,000 remaining tokens. The snapshot carries the warehouse database; the git patch does not. Clean 0/5, git diff 0/5, full snapshot 4/5. Not pooled with Figure 3.

Here, preserving the environment changes the outcome. The Git diff does not contain the database state required by the verifier, while the snapshot does. Restoring the full machine state cuts the required inference nearly in half. Snapshot restorations solved the task with just 21k–41k tokens, while clean restarts and Git-diff starts consumed significantly more tokens and still failed.



ing that migration. A new agent in a restored machine still has to notice what work remains.

Together, these experiments suggest a clear boundary for when full-machine restoration is most useful. It matters most when important progress lives outside the repository, such as in databases, installed packages, running services, generated artifacts, or other machine state that cannot be recovered from Git or source files alone.

8 Failure Analysis

The negative headline result is highly structured. We observe four recurring failure modes.

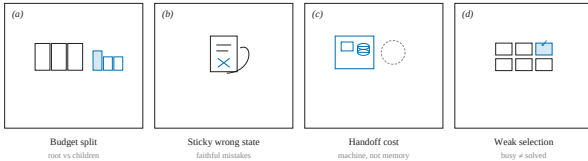


Figure 6. Why snapshot search still loses on file-centric tasks. (a) A fixed split starves the root or the children. (b) Restore is faithful to incorrect files. (c) The child inherits the machine, not the parent’s understanding. (d) Easy signals of activity are not the verifier.

Budget fragmentation. A retry receives a complete independent attempt. Snapshot search instead spends part of its budget producing the root and divides the remainder among children. A weak root creates a weak archive, while short-lived children often run out of budget before completing a repair. Our current approach uses uniform budget allocation, but this is a natural target for improvement. Adaptive budgeting strategies, learned allocation policies, or task-specific heuristics could likely balance root exploration against child continuation more effectively. We expect future work to substantially improve upon this naive distribution.

Wrong-state attractors. Restoration faithfully preserves mistakes as well as progress. If the selected checkpoint contains a plausible but incorrect implementation, the child often continues editing around that decision rather than reconsidering it. Faithful restoration can therefore make an incorrect state unusually sticky. Our current approach treats all checkpoints equally, but this is an opportunity for refinement. State quality assessment, diversity mechanisms that encourage exploration of alternative solutions, or methods to detect and escape incorrect attractors could help children break out of bad local decisions.

Handoff cost. A fresh agent does not inherit the parent’s understanding of the environment. It may receive the parent’s filesystem but still need to rediscover how the repository works, how tests should be run, and what the previous agent was attempting. In one inspected trajectory, a child consumed roughly 190,000 tokens over 26 steps, while a successful fresh retry required about 90,000 tokens over 11 steps. This suggests an opportunity for better context transfer. State summaries, environmental documentation, or explicit handoff protocols could help new agents reuse the parent’s implicit knowledge. Such mechanisms could substantially reduce the overhead of agent transitions.

Weak state selection. Deciding that a state is worth returning to is itself difficult. File edits, successful commands, or locally passing tests are imperfect proxies for actual progress. Early versions of our selector sometimes ranked package installation or claimed completion as strong evidence of progress, causing the archive to retain states that were easy to recognize but not useful to continue from. This points to a clear limitation in our ranking approach. Learned selection heuristics, task-specific progress metrics, or verifier feedback could improve the quality of archived states and help focus search on genuinely valuable checkpoints.

These failures point to a broader distinction. Snapshotting solves the state-restoration problem, but it does not solve the state-selection problem. Effective environment-level search requires both.

9 Threats to Validity

Our results cover one model tier and five screened Terminal-Bench tasks with five seeds each. The tasks were selected for intermediate baseline difficulty rather than sampled randomly, so the appropriate unit of generalization is the task rather than the individual trajectory.

Terminal-Bench is also largely file-centric. Tasks involving long-lived services, databases, caches, package installation, simulation state, or other persistent machine changes may place greater value on complete environment restoration.

Finally, we compare against independent retry rather than stronger search baselines such as best-of-N judging or SWE-Search, and we do not claim that the root/child budget allocation or snapshot-selection policy used here is optimal.

10 Related Work

Our work sits at the intersection of test-time search for software agents, iterative program repair, and state-return methods from reinforcement learning. The main distinction we study is what gets preserved between attempts: text, source code, trajectories, or the complete execution environment.

Test-time scaling for software engineering. CodeMonkeys provides an inspiring demonstration of how much software agents can gain from additional test-time computation. It samples many editing trajectories, runs model-generated tests, and selects among candidate patches. The work shows that both serial and parallel test-time compute can substantially improve software-engineering performance (Ehrlich et al., 2025). SWE-Search takes a complementary approach by organizing search with Monte Carlo tree search, value estimation, and iterative feedback to decide which software states deserve more exploration (Antoniadou et al., 2024).

Our work builds on this broader idea of spending computation more intelligently at test time. We focus on a different question. Instead of asking only how many trajectories to sample or how to score them, we ask what state a branch should resume from. In our system, a branch can inherit the complete live sandbox rather than reconstructing progress from repository state or agent context. Our results suggest that preserving more state is useful in some settings, but that good state selection and budget allocation are still essential.

Iterative repair and memory. RepairAgent is an important example of an autonomous repair system that can gather information, modify code, call tools, and validate candidate repairs through repeated interaction with a software environment (Bouzenia et al., 2024). Reflexion offers another influential approach to learning from failure. It converts feedback from previous attempts into textual reflections that guide future trials (Shinn et al., 2023).

These systems show that failed attempts can contain useful information that should not be discarded. Our work explores the same idea at the machine-state level. Textual memory or source patches may be enough when progress can be expressed in words or reconstructed from files. When progress lives in a database, installed dependency, service state, or another part of the machine, a full environment restore can preserve information that those lighter-weight methods cannot.

Agentless provides a valuable counterpoint. Xia et al.

show that a relatively simple localization, repair, and validation pipeline can compete with more elaborate autonomous agents (Xia et al., 2024). We view this as an important lesson for our own work. More machinery is only worthwhile when it preserves something that matters. On the file-centric Terminal-Bench tasks we tested, independent retries remain stronger under the matched budget. On tasks with important state outside the repository, full snapshots become much more valuable.

Returning to promising states. Our search procedure is most directly inspired by Go-Explore. In this influential line of work, Ecoffet et al. show that exploration becomes easier when an agent can remember promising states, return to them reliably, and continue exploring from there (Ecoffet et al., 2019; 2021).

We bring that intuition into coding-agent environments by changing what a saved state means. In Atari, a cell represents a compact environment state. In our setting, the equivalent object is a live software sandbox containing files, installed dependencies, process state, databases, and other artifacts created during execution.

This setting introduces a new challenge. A perfectly restored state can still be a poor place to continue from. It may contain a wrong partial solution. The next agent may not understand how that state was reached. Revisiting it also consumes tokens that could have been spent on a fresh attempt. Our experiments therefore separate two questions. Can we restore the state faithfully? Is that state actually worth returning to?

Positioning. Prior work has shown strong results from preserving generated candidates, textual experience, and source-level progress. Our work adds full machine state to that design space. We do not argue that full snapshots are always better. Instead, we study when they preserve useful information that source files and textual memory miss. Our results suggest that this distinction becomes most important when progress extends beyond the repository.

11 Conclusion

Full sandbox snapshots let coding agents return to intermediate execution states that cannot always be represented by a patch or textual summary. Our experiments show that this capability is real, but its value depends on what the environment contains and whether the system can identify states that are actually worth revisiting.

For file-centric tasks, simpler representations and fresh retrievals are often enough. When progress lives in databases, services, installed dependencies, or other machine state outside the repository, full snapshots become more compelling.

The main open problem is therefore not restoration, but selection: how can we tell when a machine state represents reusable progress rather than a dead end? Better progress signals, cheaper handoffs between agents, and tasks with richer environment state are promising directions for making environment-level search useful in practice.

References

- Antoniades, A., Örwall, A., Zhang, K., Xie, Y., Goyal, A., and Wang, W. SWE-Search: Enhancing software agents with Monte Carlo tree search and iterative refinement. arXiv:2410.20285, 2024. URL <https://arxiv.org/abs/2410.20285>.
- Bouzenia, I., Devanbu, P., and Pradel, M. RepairAgent: An autonomous, LLM-based agent for program repair. arXiv:2403.17134, 2024. URL <https://arxiv.org/abs/2403.17134>.
- Ecoffet, A., Huizinga, J., Lehman, J., Stanley, K. O., and Clune, J. Go-Explore: A new approach for hard-exploration problems. arXiv:1901.10995, 2019. URL <https://arxiv.org/abs/1901.10995>.
- Ecoffet, A., Huizinga, J., Lehman, J., Stanley, K. O., and Clune, J. First return, then explore. *Nature*, 590:580–586, 2021.
- Ehrlich, R., Brown, B., Juravsky, J., Clark, R., Ré, C., and Mirhoseini, A. CodeMonkeys: Scaling test-time compute for software engineering. arXiv:2501.14723, 2025. URL <https://arxiv.org/abs/2501.14723>.
- Harbor Framework Team. Harbor: A framework for evaluating and optimizing agents and models in container environments, 2026. URL <https://doi.org/10.5281/zenodo.20953922>. Version v0.16.1. doi: 10.5281/zenodo.20953922.
- Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning. arXiv:2303.11366, 2023. URL <https://arxiv.org/abs/2303.11366>.
- The Terminal-Bench Team. Terminal-Bench: A benchmark for AI agents in terminal environments, April 2025. URL <https://github.com/laude-institute/terminal-bench>.
- Xia, C. S., Deng, Y., Dunn, S., and Zhang, L. Agentless: Demystifying LLM-based software engineering agents. arXiv:2407.01489, 2024. URL <https://arxiv.org/abs/2407.01489>.